CS 216: Bitcoin Transaction Lab Assignment

Team Members :

1. Mane Abhishek Ganesh (240001043)
2. Chetan Verma (240001022)
3. Dhoke Vinod Eknath (240001025)
4. Sahil Hedaoo (240041032)

Introduction :

This assignment focuses on understanding how Bitcoin transactions work internally by creating and analyzing transactions using two address formats:

- **Legacy addresses (P2PKH)**
- **SegWit addresses (P2SH-P2WPKH)**

The experiments are conducted using Bitcoin Core in regtest mode, allowing transactions to be created and tested locally without using real cryptocurrency. Through these experiments, we analyze how transactions are constructed, how scripts validate transactions, and how SegWit improves transaction efficiency compared to legacy transaction formats.

Part 1 – Legacy Address Transactions (P2PKH) :

In this part of the experiment, Bitcoin transactions were created using **Legacy Pay-to-Public-Key-Hash (P2PKH) addresses**.

1. Transaction Workflow

Three legacy Bitcoin addresses were generated using the wallet:

- Address A
- Address B
- Address C

The experiment consisted of two transactions forming the following workflow:

A → B → C

Transaction 1: Address A → Address B

Coins were sent from Address A to Address B.

Transaction ID:
8a0cd0d4d4370b8b77f02d1721cce45bacdafe6c0a34263e6581467cc0c0da72

This transaction created a **UTXO (Unspent Transaction Output)** for Address B. The output from this transaction became the input for the next transaction.

Transaction 2: Address B → Address C

The output from the previous transaction (A → B) was used as an input to create a second transaction from **Address B to Address C**.

Transaction ID:1e55e087528e9b9e3cff105820b2b5b5215002aca57578d1516ef42dff6236c1

This demonstrates the **UTXO model used in Bitcoin**, where outputs from previous transactions serve as inputs for new transactions.

2. Decoded Scripts of Transactions

After creating the raw transactions, they were decoded using the `decoderawtransaction` command to analyze their script structure.

Decoded Script for Transaction A → B

The output script (`scriptPubKey`) for Address B was decoded and displayed as follows:

OP_DUP OP_HASH160 c7702fd14c8f4480d010f3bfd8dbe906816ee87c OP_EQUALVERIFY OP_CHECKSIG

This script represents a **P2PKH locking script**, which specifies the conditions required to spend the output.

```
(venv) (base) abhishek@Abhisheks-Laptop Part1_Legacy % python part1.py
--- Generating Addresses ---
Address A: myni6nedNrcYtQPnBGZPbfYUC9arp8zteH
Address B: myhVA6vTthoNhPHSHtMaoYnP6kVM3X9RrP
Address C: mrAQfcAgEMqw46gzpjJkiCPXZFWMC8GMqg

--- Funding Address A ---
Mined 101 blocks. Address A is funded.

--- Creating TX: A -> B ---
TXID A -> B: 8a0cd0d4d4370b8b77f02d1721cce45bacdafe6c0a34263e6581467cc0c0da72

[Decoded A->B] ScriptPubKey for Address B:
{
  "asm": "OP_DUP OP_HASH160 c7702fd14c8f4480d010f3bfd8dbe906816ee87c OP_EQUALVERIFY OP_CHECKSIG",
  "desc": "addr(myhVA6vTthoNhPHSHtMaoYnP6kVM3X9RrP)#7ns0ekm",
  "hex": "76a914c7702fd14c8f4480d010f3bfd8dbe906816ee87c88ac",
  "address": "myhVA6vTthoNhPHSHtMaoYnP6kVM3X9RrP",
  "type": "pubkeyhash"
}
```

Decoded Script for Transaction B → C

Similarly, the transaction from **Address B to Address C** was decoded to analyze the unlocking script.

The input script (`scriptSig`) contains:

0335e3ea5c82c99466c4f4fe1380e4a6adee77bb443236977227163bec6c208364

```
--- Creating TX: B -> C ---
TXID B -> C: 1e55e087528e9b9e3cff105820b2b5b5215002aca57578d1516ef42dff62361

[Decoded B->C] ScriptSig (Unlocking Script) from B:
{
  "asm": "304402206b444d67d1de5b8fc1e556d7709a546fd6c2bd6aeaf1d33e97e98864ac0c5b8002201252d75753ba4509471019d642e9151c0e16f65653b763f4f2e3c4e9c0bbc39a[ALL] 0335e3ea5c82c99466c4f4fe1380e4a6adee77bb443236977227163bec6c208364",
  "hex": "47304402206b444d67d1de5b8fc1e556d7709a546fd6c2bd6aeaf1d33e97e98864ac0c5b8002201252d75753ba4509471019d642e9151c0e16f65653b763f4f2e3c4e9c0bbc39a01210335e3ea5c82c99466c4f4fe1380e4a6adee77bb443236977227163bec6c208364"
}

--- Script Analysis (Part 1: Legacy P2PKH) ---
Size: 225 bytes
vSize: 225 vbytes
Weight: 900 wu

Done. Check the outputs to use in btcdeb!
(venv) (base) abhishek@Abhisheks-Laptop Part1_Legacy %
```

3. Structure of Challenge and Response Scripts

Bitcoin transactions use a **challenge-response mechanism** to validate whether a transaction is authorized.

Challenge Script (Locking Script)

The **locking script** (scriptPubKey) from the output of the first transaction acts as a challenge.

Structure:

OP_DUP OP_HASH160 c7702fd14c8f4480d010f3bfd8dbe906816ee87c OP_EQUALVERIFY OP_CHECKSIG

This script enforces the following conditions:

1. Duplicate the public key.
2. Hash the duplicated public key.
3. Compare the result with the stored public key hash.
4. Verify the signature using the provided public key.

If all conditions are satisfied, the transaction is considered valid.

Response Script (Unlocking Script)

The **unlocking script** (scriptSig) provides the data required to satisfy the locking script.

Structure:

0335e3ea5c82c99466c4f4fe1380e4a6adee77bb443236977227163bec6c208364

The unlocking script supplies:

- A valid digital **signature**
- The corresponding **public key**

When combined with the locking script, the Bitcoin script interpreter executes both scripts sequentially.

If the final result of the script execution evaluates to **TRUE**, the transaction is accepted as valid.

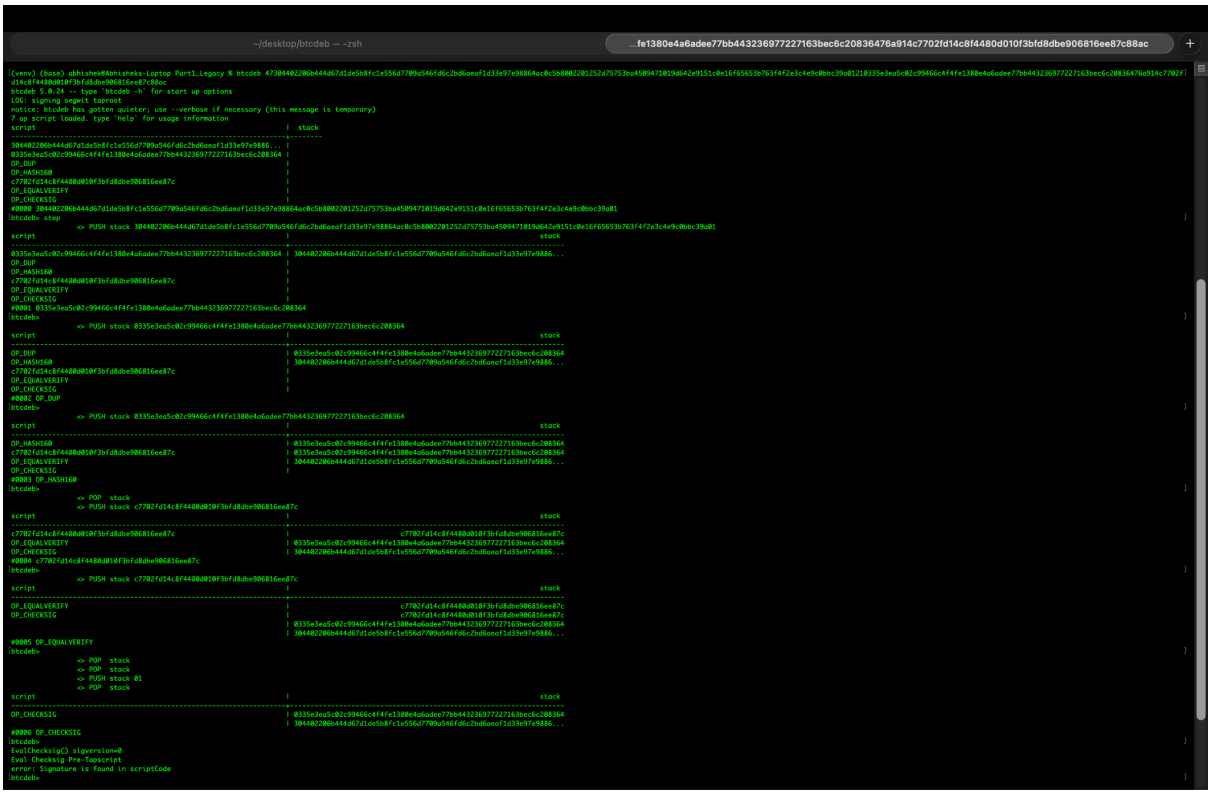
4. Script Validation using Bitcoin Debugger

To verify the correctness of the scripts, the transaction scripts were executed using the **Bitcoin Script Debugger (btcdeb)**.

The debugger allows step-by-step execution of Bitcoin scripts, showing how the stack changes during execution.

The execution confirmed that:

- The provided signature matched the public key.
- The public key hash matched the address specified in the locking script.
- The final script evaluation returned **TRUE**, confirming that the transaction was valid.



Part 2 – SegWit Address Transactions (P2SH-P2WPKH)

In this part of the experiment, Bitcoin transactions were created using **SegWit addresses in the P2SH-P2WPKH format**. SegWit (Segregated Witness) is an upgrade to the Bitcoin protocol that separates signature data from the main transaction structure. This improves transaction efficiency and reduces the effective transaction size.

1. Transaction Workflow

Three SegWit addresses were generated using the wallet:

- Address A'
- Address B'
- Address C'

Two transactions were performed forming the following workflow:

A' → B' → C'

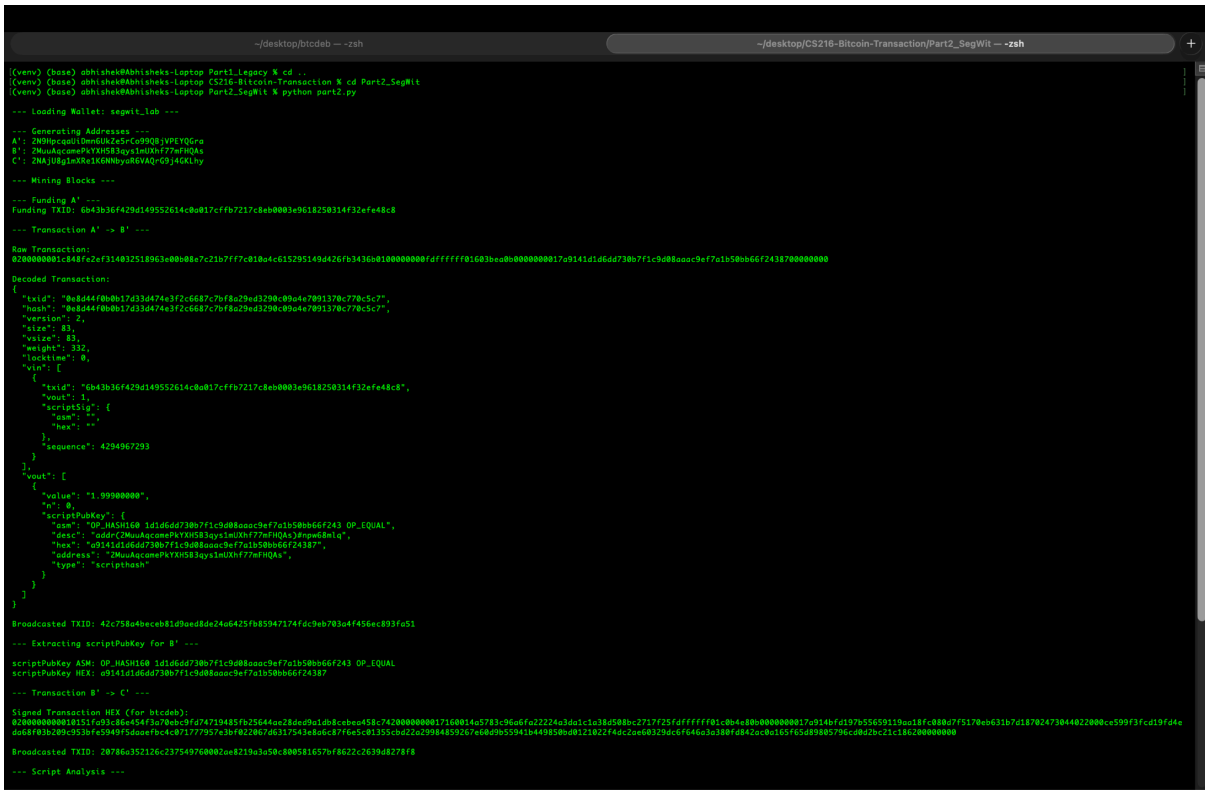
Transaction 1: Address A′ → Address B′

Coins were sent from **Address A′ to Address B′**.

Transaction ID:

txid: 42c758a4beceb81d9aed8de24a6425fb85947174fdc9eb703a4f456ec893fa51

This transaction created a **UTXO for Address B′**, which was later used as the input for the next transaction.



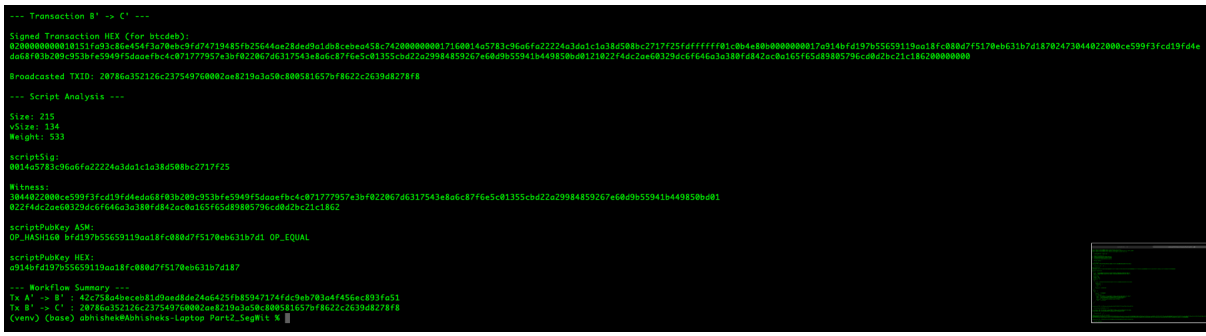
Transaction 2: Address B′ → Address C′

The output from the previous transaction was used as the input for a second transaction from **Address B′ to Address C′**.

Transaction ID:

txid: 20786a352126c237549760002ae8219a3a50c800581657bf8622c2639d8278f8

This demonstrates how SegWit transactions also follow the **UTXO model**, where outputs from one transaction serve as inputs for another transaction.



2. Decoded Scripts of Transactions

The raw transactions were decoded using the `decoderawtransaction` command to analyze the script structures involved in SegWit transactions.

Decoded Script for Transaction A′ → B′

The output script (`scriptPubKey`) for the SegWit address was decoded and typically appears as:

OP_HASH160 <Script Hash> OP_EQUAL

This indicates that the transaction is using **P2SH (Pay-to-Script-Hash)** to wrap the SegWit script.

Decoded Script for Transaction B′ → C′

In the second transaction, the unlocking data includes the **redeem script and witness data**, which contain the signature and public key required to validate the transaction.

Structure observed in the decoded transaction:

scriptSig → redeem script

witness → <signature> <public key>

3. Structure of Challenge and Response Scripts

Similar to legacy transactions, SegWit transactions also use a **challenge-response mechanism** for validation.

However, the structure differs because signature data is separated into the **witness field**.

Challenge Script (Locking Script)

The locking script in a P2SH-P2WPKH transaction typically appears as:

OP_HASH160 <Script Hash> OP_EQUAL

This script ensures that the redeem script provided in the unlocking script matches the expected script hash.

The redeem script then defines the conditions required to spend the output.

Response Script (Unlocking Script)

The unlocking data consists of two parts:

scriptSig

<redeem script>

witness

<signature> <public key>

The redeem script corresponds to a **SegWit P2WPKH script**, which verifies that:

- The public key hash matches the address
- The signature is valid for the transaction

When executed together, the script components validate the ownership of the funds.

Execution flow:

scriptSig + scriptPubKey + witness

If the execution evaluates to **TRUE**, the transaction is considered valid.

4. Script Validation using Bitcoin Script Debugger

To verify the correctness of the SegWit transaction scripts, the scripts were executed using the **Bitcoin Script Debugger (btcdeb)**.

The debugger allows step-by-step execution of the script, showing how the stack changes during each operation.

During execution, the debugger confirmed that:

- The redeem script matched the script hash in the locking script.
- The provided public key corresponded to the address.
- The digital signature was valid.
- The final script evaluation returned **TRUE**, confirming successful validation of the transaction.

Part 3 – Analysis and Explanation

This section presents a comparative analysis of the **Legacy (P2PKH)** transactions performed in Part 1 and the **SegWit (P2SH-P2WPKH)** transactions performed in Part 2.

1. Transaction Size Comparison

One of the main improvements introduced by SegWit is the reduction in effective transaction size. To analyze this difference, the sizes of the transactions created in Part 1 and Part 2 were examined.

Transaction Type	Transaction	Size (Bytes)	Weight	Virtual Size (vBytes)
Legacy (P2PKH)	A → B	225	900	225
Legacy (P2PKH)	B → C	225	900	225
SegWit (P2SH-P2WPKH)	A' → B'	215	533	134
SegWit (P2SH-P2WPKH)	B' → C'	215	533	134

From the results, it can be observed that SegWit transactions typically have **smaller virtual size (vbytes)** compared to legacy transactions.

This occurs because SegWit separates signature data from the main transaction structure, reducing the portion of the transaction counted toward the block size limit.

2. Script Structure Comparison

Another important difference between Legacy and SegWit transactions lies in the structure of their scripts.

Legacy Transaction Script (P2PKH)

In a Legacy transaction, the locking script (challenge script) stored in the output is:

"OP_DUP OP_HASH160 c7702fd14c8f4480d010f3bfd8dbe906816ee87c OP_EQUALVERIFY OP_CHECKSIG"

The unlocking script (response script) provided in the input is:

<Signature> <Public Key>

During validation, the unlocking script and locking script are executed together:

scriptSig + scriptPubKey

If the final result of the execution evaluates to **TRUE**, the transaction is considered valid.

SegWit Transaction Script (P2SH-P2WPKH)

In SegWit transactions, the script structure is slightly different.

The locking script stored in the output appears as:

OP_HASH16Ø 1d1d6dd73Øb7f1c9d08aaac9ef7a1b50bb66f243 OP_EQUAL

The unlocking data is divided into two parts : **scriptSig and witness.**

The witness field contains the signature and public key required to satisfy the redeem script. During validation, the redeem script and witness data are used to verify the transaction.

3. Why SegWit Transactions Are Smaller

SegWit transactions are smaller because the signature data is separated from the main transaction structure.

In Legacy transactions, all data—including signatures—is included in the base transaction size. However, SegWit introduces the concept of **transaction weight**, which calculates the size of transactions differently.

The weight of a transaction is calculated using the formula:

Transaction Weight = (Base Size × 4) + Witness Size

The effective size used for fee calculation is called **virtual size (vbytes)**:

Virtual Size (vbytes) = Transaction Weight / 4

Because witness data is counted at a lower weight than the base transaction data, SegWit transactions have a smaller effective size compared to legacy transactions.

4. Benefits of SegWit Transactions

SegWit provides several advantages over legacy transaction formats:

1. Reduced transaction size
   Signature data is separated into the witness field, reducing the effective transaction size.

2. Lower transaction fees
   Since fees are calculated based on virtual size (vbytes), smaller transactions result in lower fees.

3. Better scalability
   SegWit allows more transactions to fit within a block, improving the scalability of the Bitcoin network.

4. Fixes transaction malleability
   SegWit prevents certain types of transaction modification, improving the reliability of advanced features such as the Lightning Network.